Save and Load model and parameters

高怡宣
陽明大學 生物醫學影像暨放射科學系

jupyter Iris-save_model_parameters Last Checkpoint: 幾秒前 (autosaved)

File Edit View Insert Cell Kernel Widgets Help

Run Markdown

Keras 儲存與載入訓練好的模型或參數教學

Iris, sepal: 萼片, petal: 花瓣

data from: https://www.kaggle.com/uciml/iris/data

code modified from: https://www.kaggle.com/pratik2901/sequential-ann-and-visualization

code modified from: https://morvanzhou.github.io/tutorials/machine-learning/keras/3-1-save/

Save & reload 保存提取, 作者: Alice 编辑: 莫烦 2016-10-30

In [2]:
```
np.random.seed(3)
```

In [3]:
```
iris = pd.read_csv("Iris.csv")
```

In [4]:
```
dataset = iris.values
X = dataset[:,1:5].astype(float)
#print(X[0:5,:])
```

In [5]:
```
# convert text into vectors, method #2
Y = dataset[:,5]
encoder = LabelEncoder()
encoder.fit(Y)
encoded_Y = encoder.transform(Y)
# print(Y[0:5])
# print(encoded_Y)
```

In [6]:
```
answer_Y = np_utils.to_categorical(encoded_Y)
# print(answer_Y[0:5])
```

(2) Sequential Based ANN

```
def baseline_model():
    test = optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999, \
                           epsilon=None, decay=0.0, amsgrad=False)
    model = Sequential()
    model.add(Dense(8, input_dim=4, activation='relu'))
    model.add(Dense(3, activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer=test, metrics=['accuracy'])
    return model
```

[8]:
```
model = baseline_model()
model.summary()
```

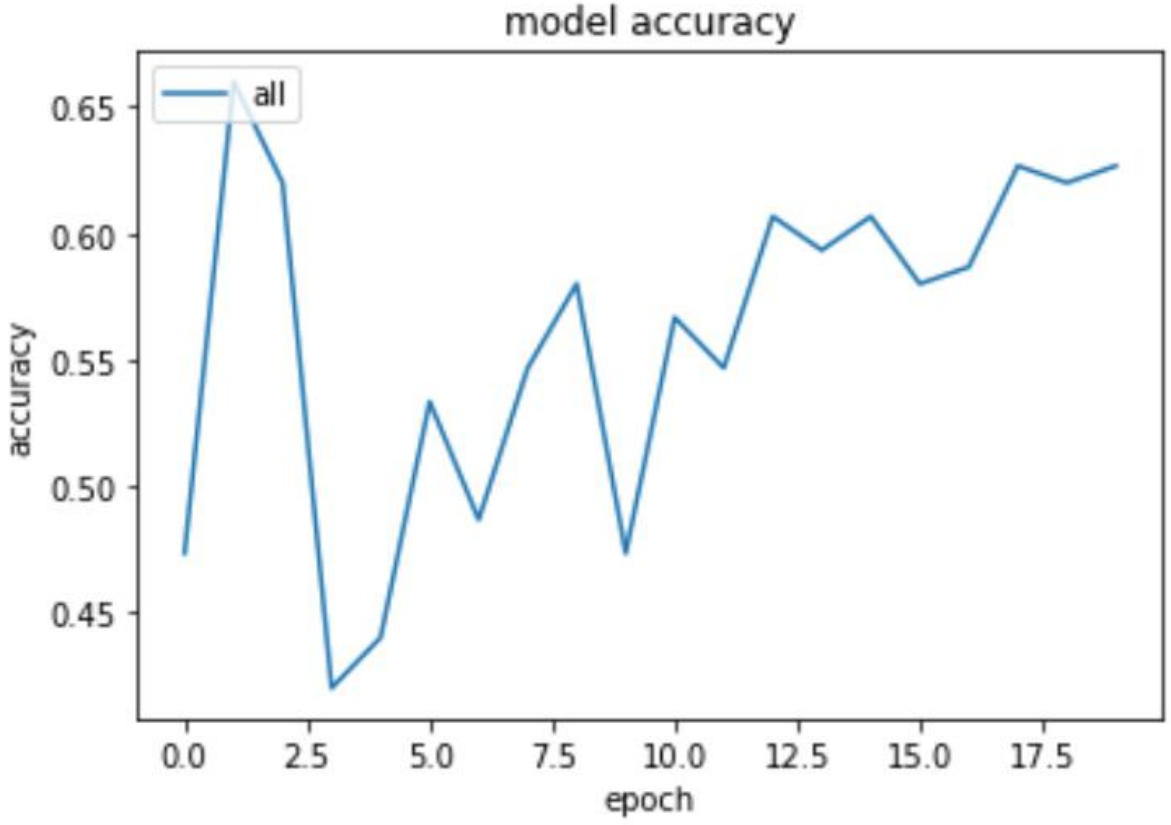
```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 8)                 40
_________________________________________________________________
dense_2 (Dense)              (None, 3)                 27
=================================================================
Total params: 67
Trainable params: 67
Non-trainable params: 0
_________________________________________________________________
```

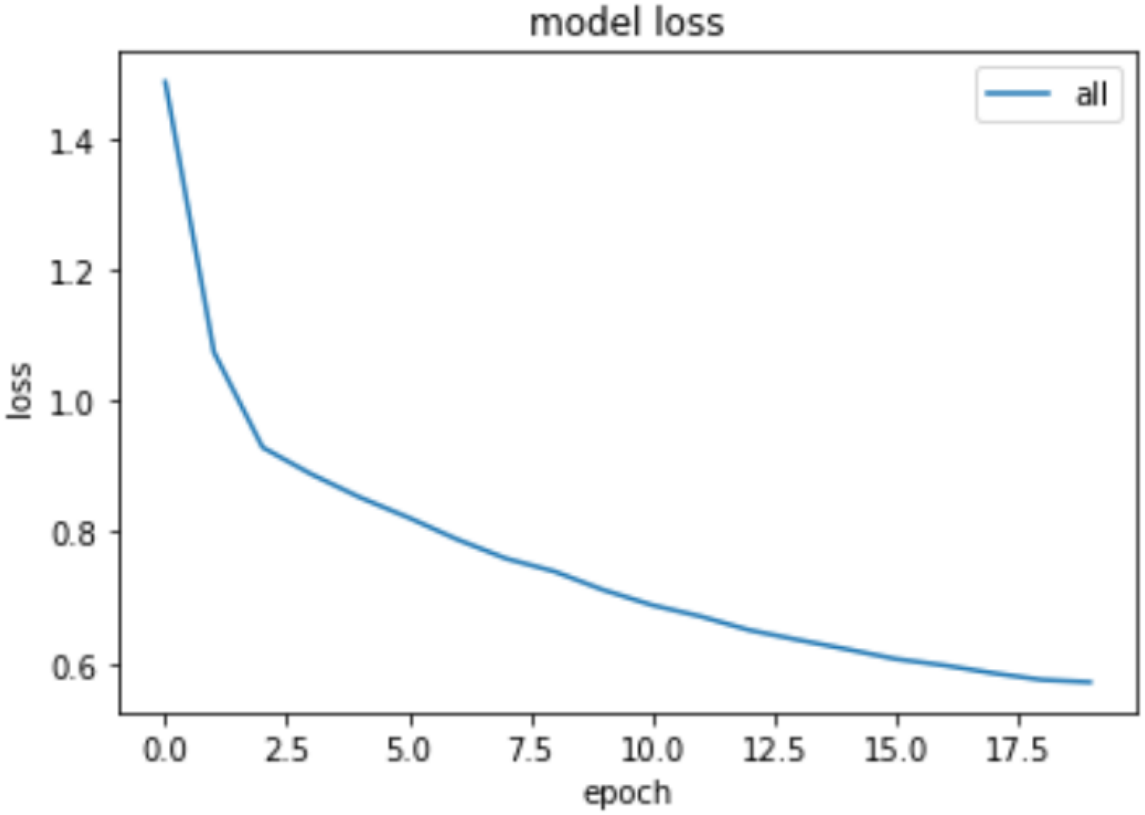
[9]:
```
history = model.fit(X,answer_Y,epochs=20,batch_size=5)
```

```
Epoch 1/20
150/150 [==============================] - 0s 1ms/step - loss: 1.4857 - acc: 0.4733
Epoch 2/20
```

In [10]:
```
# summarize history for accuracy
plt.plot(history.history['acc'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['all'], loc='upper left')
plt.savefig('acc.png')
plt.show()

# summarize history for loss
plt.plot(history.history['loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['all'], loc='upper right')
plt.savefig('loss.png')
plt.show()
```


model accuracy
all
accuracy
0.65
0.60
0.55
0.50
0.45
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
epoch


model loss
all
loss
1.4
1.2
1.0
0.8
0.6
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
epoch

(3) 今天學習如何保存神經網絡，這樣以後想要用的時候直接提取就可以。

保存模型

訓練完模型之後，可以打印一下預測的結果，接下來就保存模型。

保存的時候只需要一行代碼**model.save**，再給它加一個名字就可以用**h5**的格式保存起來。

這裡註意，需要已經安裝了**HDF5**這個模塊。

保存完模型之後，刪掉它，後面可以來比較是否成功的保存。

```
# 將模型儲存至 HDF5 檔案中
print('test before save: ', model.predict(X[0:2]))
model.save('my_model.h5')   # HDF5 file, you have to pip3 install h5py
```

```
test before save:  [[0.8724854  0.06851488 0.05899969]
 [0.7983357  0.10684371 0.09482066]]
```

```
del model  # deletes the existing model
```

(4) 導入模型並應用

導入保存好的模型，再執行一遍預測，與之前預測的結果比較，可以發現結果是一樣的。

```
# load HDF5 file
from keras.models import load_model
model = load_model('my_model.h5')
print('test after load: ', model.predict(X[0:2]))
```

```
test after load:  [[0.8724854  0.06851488 0.05899969]
 [0.7983357  0.10684371 0.09482066]]
```

```
model.summary()
```

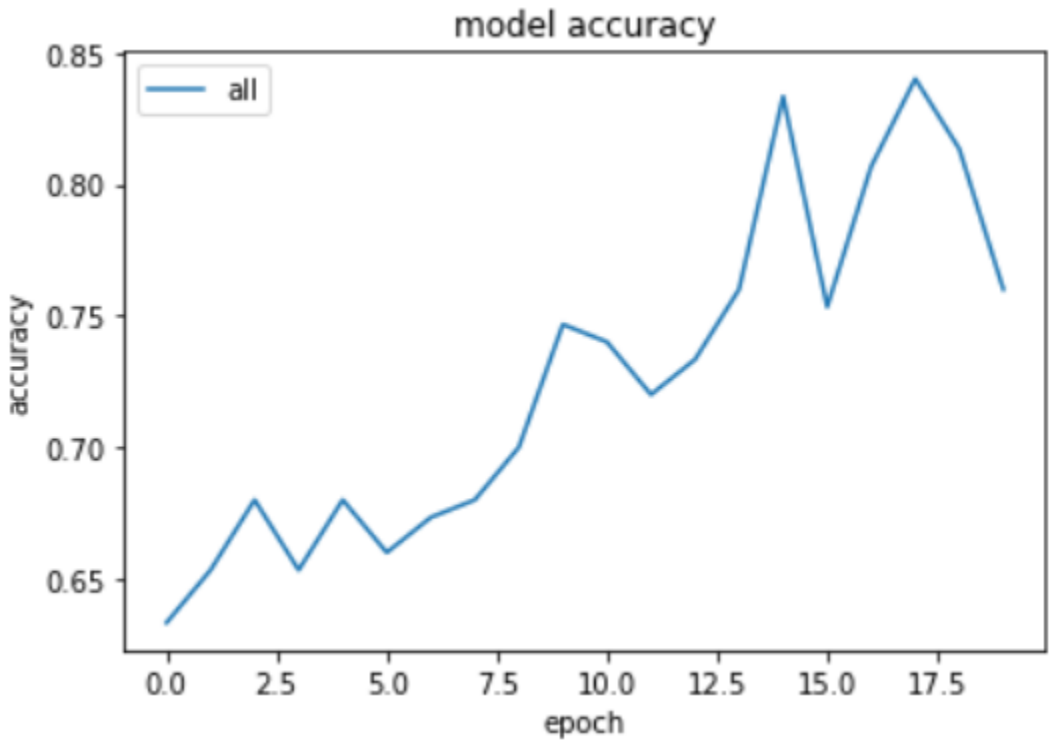
```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 8)                 40
_________________________________________________________________
dense_2 (Dense)              (None, 3)                 27
=================================================================
Total params: 67
Trainable params: 67
Non-trainable params: 0
_________________________________________________________________
```

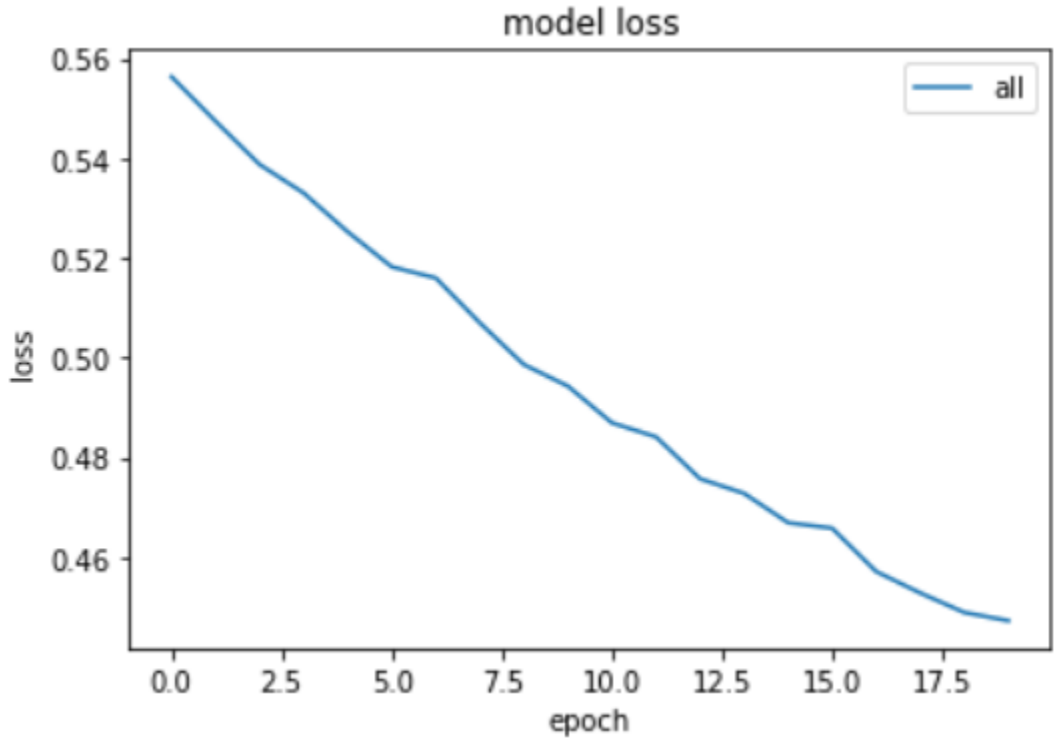
```
history = model.fit(X,answer_Y,epochs=20,batch_size=5)
```

```
Epoch 1/20
150/150 [==============================] - 0s 732us/step - loss: 0.5563 -
acc: 0.6333
Epoch 2/20
150/150 [==============================] - 0s 127us/step - loss: 0.5473 -
acc: 0.6533
Epoch 3/20
150/150 [==============================] - 0s 167us/step - loss: 0.5387 -
acc: 0.6800
```

```
Epoch 20/20
150/150 [==============================] - 0s 133us/step - loss: 0.4472 - acc:
0.7600
```

[16]:
```
# summarize history for accuracy
plt.plot(history.history['acc'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['all'], loc='upper left')
plt.savefig('acc.png')
plt.show()
```


model accuracy
all
accuracy
0.85
0.80
0.75
0.70
0.65
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
epoch


model loss
all
loss
0.56
0.54
0.52
0.50
0.48
0.46
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
epoch

(5) 另外還有其他保存模型並調用的方式，

第一種是只保存權重而不保存模型的結構。

[17]:
```
# save and load weights
model.save_weights('my_model_weights.h5')
model.load_weights('my_model_weights.h5')
```

第二種是用**model.to_json**保存完結構之後，然後再去加載這個**json_string**。

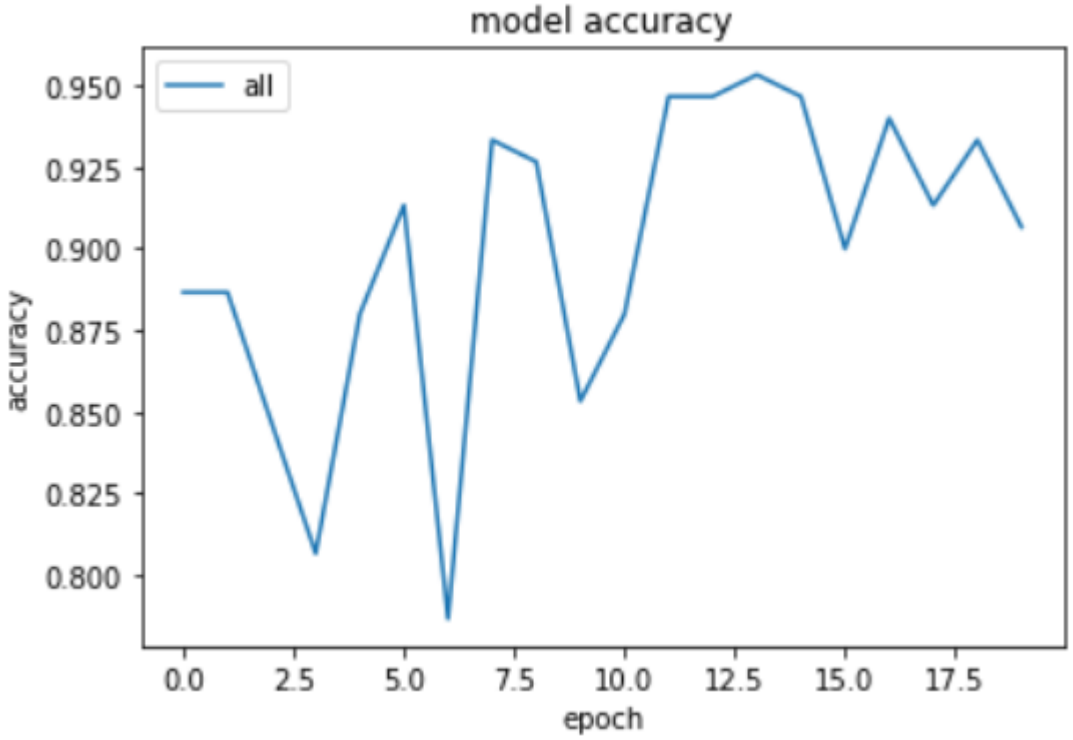
In [18]:
```
# save and load fresh network without trained weights
from keras.models import model_from_json
json_string = model.to_json()
model = model_from_json(json_string)
model.compile(loss='categorical_crossentropy', optimizer='Adam', m
model.load_weights('my_model_weights.h5')
```

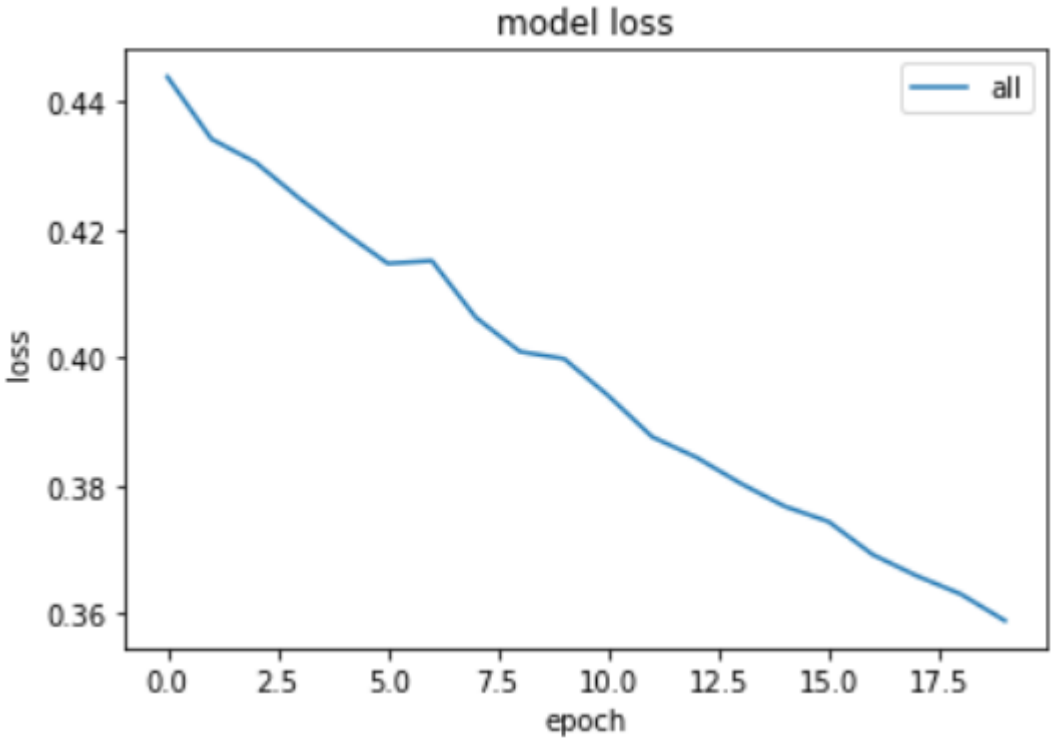
[19]:
```
model.summary()
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense_1 (Dense)              (None, 8)                 40
_________________________________________________________________
dense_2 (Dense)              (None, 3)                 27
=================================================================
Total params: 67
Trainable params: 67
Non-trainable params: 0
_________________________________________________________________
```

[20]:
```
history = model.fit(X,answer_Y,epochs=20,batch_size=5)
```

```
Epoch 1/20
150/150 [==============================] - 0s 1ms/step - loss: 0.4439 - a
cc: 0.8867
Epoch 2/20
150/150 [==============================] - 0s 140us/step - loss: 0.4341 -
acc: 0.8867
Epoch 3/20
150/150 [==============================] - 0s 140us/step - loss: 0.4305 -
acc: 0.8467
```


model accuracy
all
accuracy
0.950
0.925
0.900
0.875
0.850
0.825
0.800
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
epoch


model loss
all
loss
0.44
0.42
0.40
0.38
0.36
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
epoch

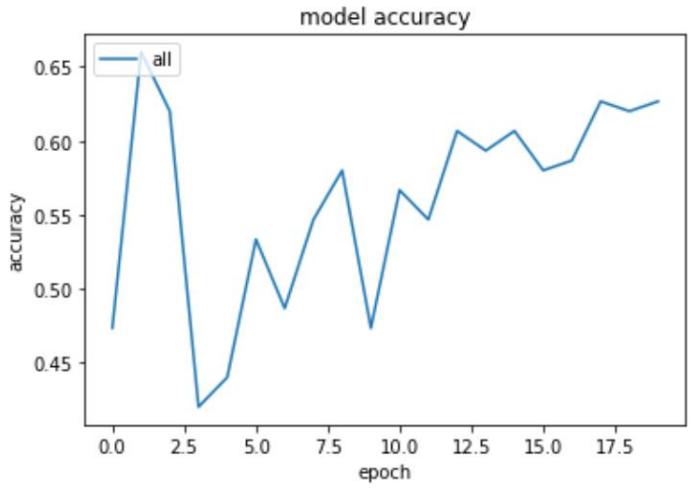
In [22]:
```
scores = model.evaluate(X,answer_Y)
```

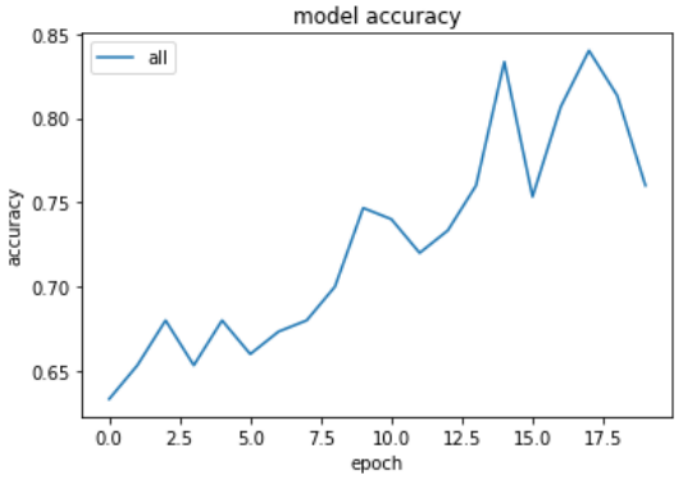
```
150/150 [==============================] - 0s 305us/step
```

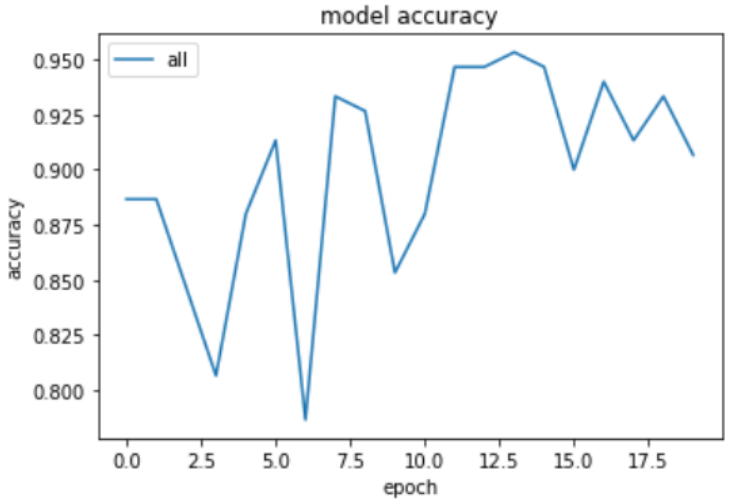
In [23]:
```
print("Accuracy:%.2f%%"%(scores[1]*100))
```

```
Accuracy:92.00%
```

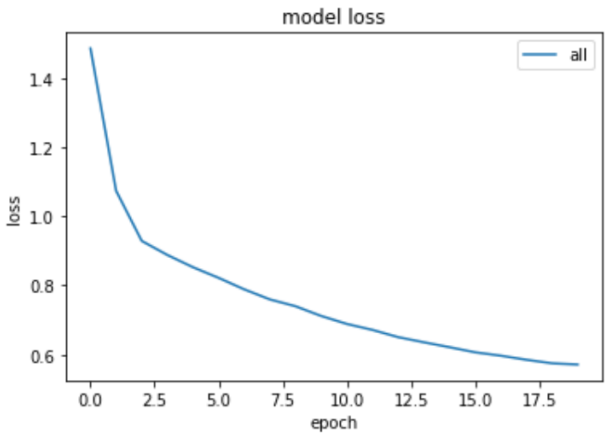
Accuracy of the 3 runs:

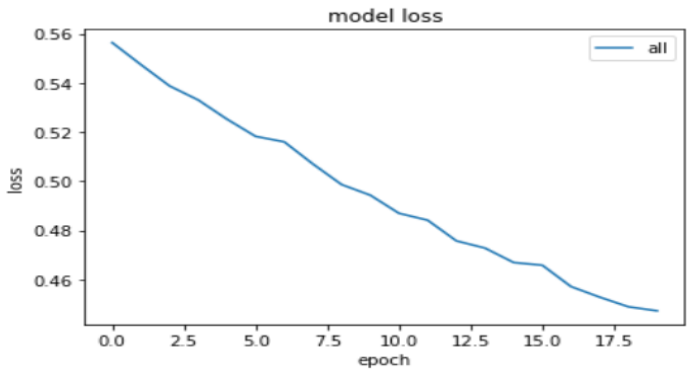


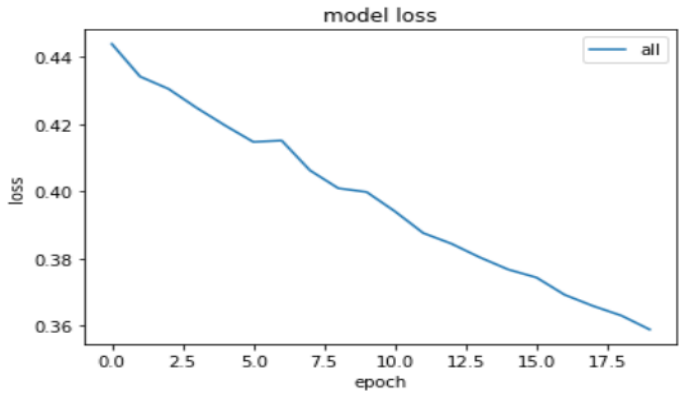




Loss of the 3 runs:







Thank you for your attention